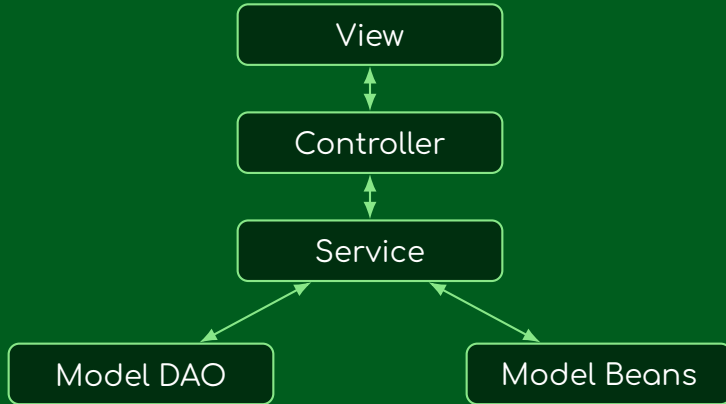


Green Refactoring of the 2chance Web Application: An Empirical Analysis of Energy Patterns and Performance

Sustainable Software Engineering
Green-in-Software Project

2Chance Web Application



View

HTML, CSS

INTERFACCIA UTENTE (UI)

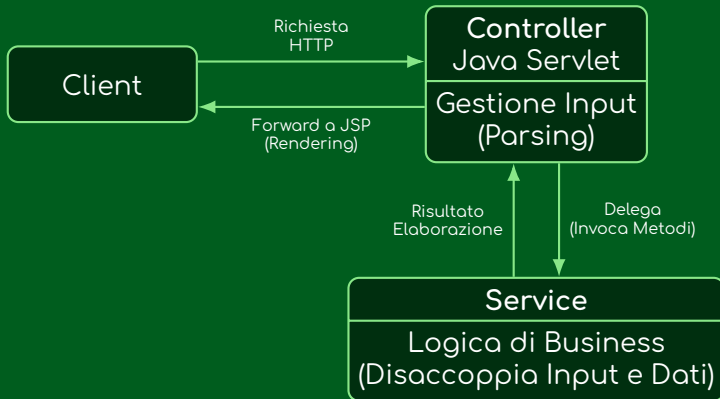
JavaScript

INTERATTIVITÀ LATO CLIENT E
ACQUISIZIONE DEGLI INPUT DELL'UTENTE

JavaServer Pages (JSP)

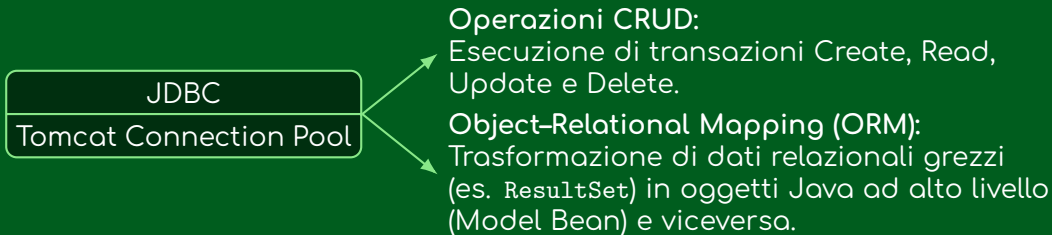
VISUALIZZARE DINAMICAMENTE DATI
FORNITI DAL CONTROLLER

Controller + Service



Model Data Access Object (DAO)

“Astrae e incapsula tutto l’accesso al database che contiene i dati persistenti.”



Software Dependability Improvements

Unit & Integration Testing

JUnit 5 → Automated Tests
Mockito → Mock Dipendenze Esterne
JaCoCo → Code Coverage
Pitest → Mutation Testing

Metodi Formali

JML → Specifica Formale
OpenJML → Verifica Formale

Deployment

Docker → Containerizzazione App
Docker Compose → Ambiente Multi-Container

GitHub Actions

GitGuardian → Secret Detection
Snyk → Software Composition Analysis
SonarQube Cloud → Static Analysis

Backend Energy Sustainability

Backend Energy Consumption

*Componente che compie il maggior sforzo computazionale →
Componente che consuma più energia.*

Componenti a maggior consumo:

- Gestione delle Interfacce (Controller)
- Business Logic (Service)
- Interrogazioni del Database (Model DAO | DBMS)

Backend Green Refactoring Plan

- Individuare tramite analisi statica e revisione del codice le porzioni di codice meno efficienti.
- Procedere al refactoring per ridurre lo spreco di risorse.
- Misurare il consumo energetico effettivo durante l'uso prima e dopo il refactoring.

Rule-Based Eco-Design Static Analysis



Docker

Poiché il progetto è containerizzato, l'esecuzione locale di SonarQube (in un container dedicato) in parallelo all'app risulta estremamente efficiente.

GitHub Actions

Poiché Creedengo opera come plugin per SonarQube, si integra direttamente nella pipeline di Static Application Security Testing (SAST) esistente.

Configurazione con Script Automatico di SonarQube con il Plugin Creedengo

1. Validazione dell'Ambiente

Verifica dell'esecuzione attiva del demone Docker sul sistema.



2. Scaricamento del Plugin Creedengo

Controllo della presenza locale del plugin ed eventuale download dinamico da GitHub.



3. Provisioning di SonarQube

Avvio di SonarQube in un container Docker configurando la corretta esposizione delle porte.



4. Iniezione del Plugin Creedengo

Copia del plugin all'interno del container, nella cartella riservata alle estensioni.



5. Riavvio ed Inizializzazione

Riavvio del container per forzare il caricamento del plugin e registrare le regole di eco-design.

Selezione delle Regole di Eco-Design

Massimizzare il Ritorno Ecologico
delle Attività di Refactoring

Ridurre il Rumore nelle
Metriche di Telemetria Statica

11 Regole Fondamentali del Ruleset Creedengo Java Selezionate

Regole Scelte per **Pertinenza Tecnologica** con l'Architettura di 2Chance Web App
*(App monolitica basata su Java Servlets e interazioni DB
tramite JDBC/DAO nativi, senza framework pesanti come Spring)*

Scansione iniziale (*Baseline*): rilevate complessivamente **64 violazioni di eco-design**
distribuite unicamente su 4 delle 11 regole (anti-pattern strutturali).